QoS Traffic Management System — Complete Project Documentation

> **Project #5** · Computer Networks Mini-Project · SEM-3
> **Protocol**: UDP · **Framework**: Ryu SDN + Mininet + Open vSwitch

Team Members

Name	SRN
Athul Jenson	PES2UG25CS633
Ananya KG	PES2UG25CS630
Vishwaraj	PES2UG25CS607

Table of Contents

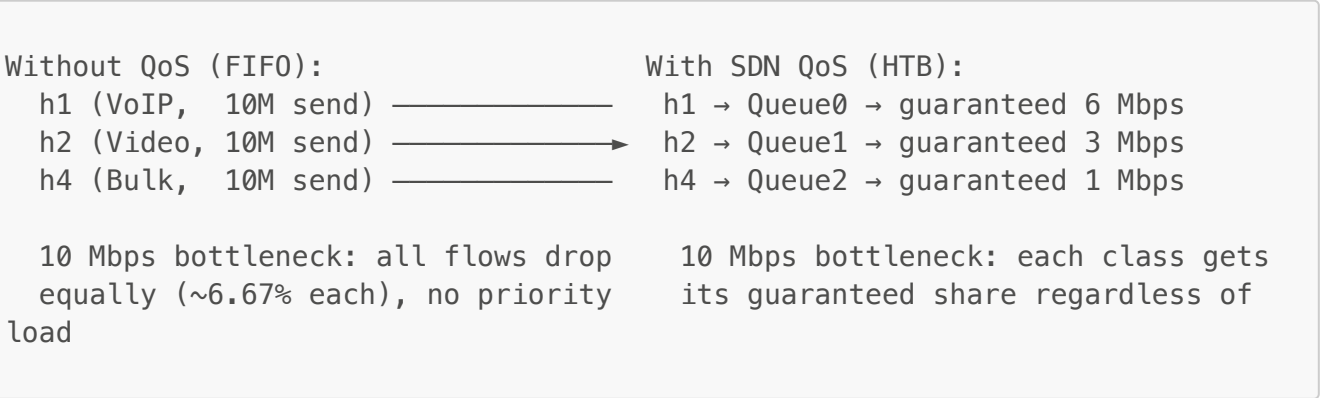
1. Project Overview
2. System Architecture
3. File-by-File Code Reference
4. Data Flow & Component Interaction
5. Key Design Decisions
6. Theoretical Background
7. Experimental Results
8. Running the Project
9. Troubleshooting Reference

---

1. Project Overview

This project designs and evaluates an **SDN-driven QoS Traffic Management System** in Mininet. The core problem addressed is *link congestion fairness*: in a standard FIFO network, a 10 Mbps bottleneck link cannot distinguish VoIP traffic from bulk file transfers. The SDN controller solves this by programming OVS switches to enforce bandwidth guarantees per traffic class.

Core Problem

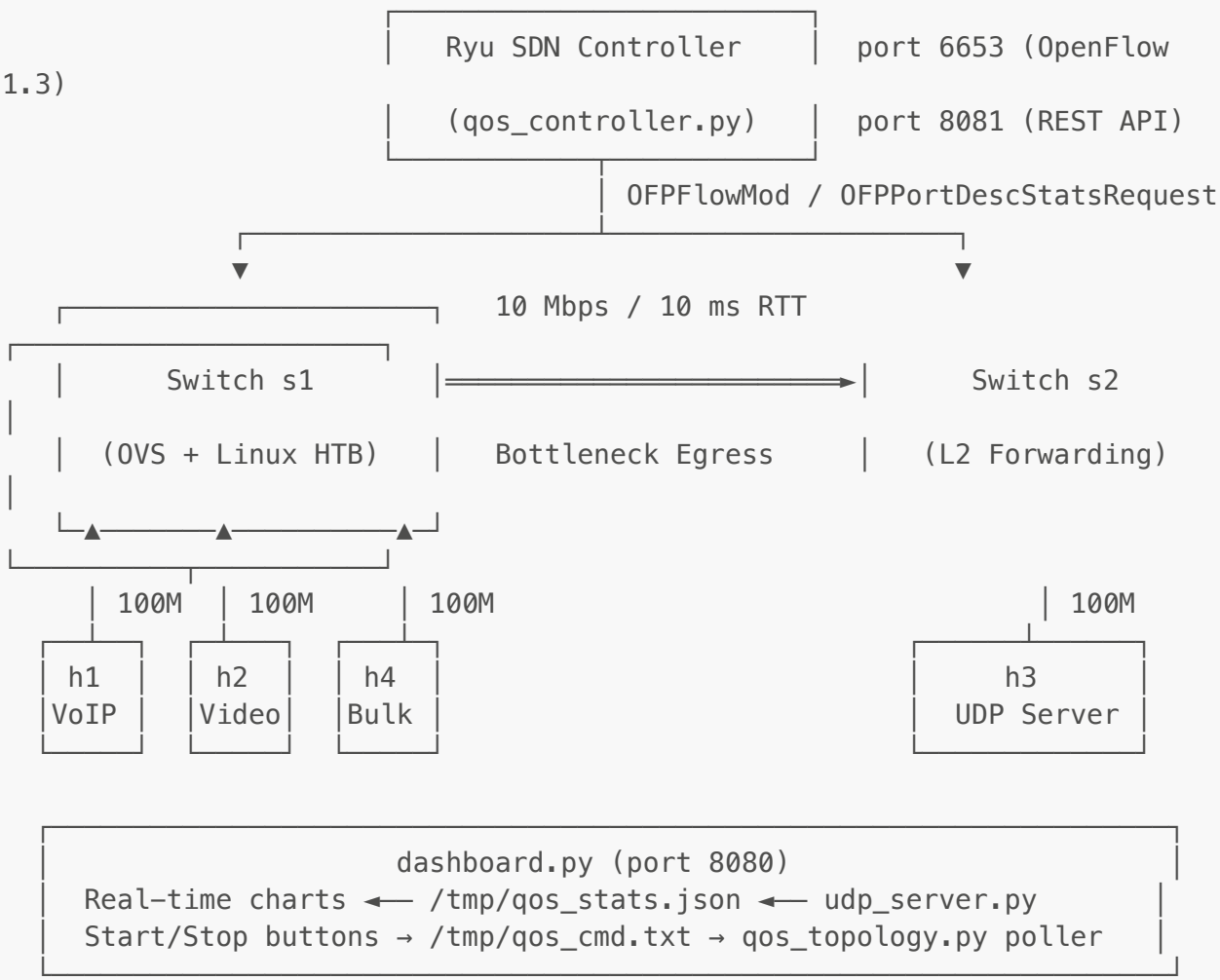


Key Figures

Metric	Value
Bottleneck link capacity	10 Mbps
Total injected load	30 Mbps (300% saturation)
Number of traffic classes	3 (High / Medium / Best-Effort)
Dynamic swap time	t = 20 s into experiment
Measured Jain's Fairness Index	**0.985** (near-optimal = 1.0)

---

2. System Architecture



QoS Priority Matrix

Host	IP	Traffic Simulation	DSCP / ToS	OVS Queue	Phase 1 (0–20s)	Phase 2 (20s+)
h1	10.0.0.1	VoIP / High-Priority	DSCP 46 (EF)	Queue 0	**6.0 Mbps**	1.0 Mbps (Demoted)

Host	IP	Traffic Simulation	DSCP / ToS	OVS Queue	Phase 1 (0–20s)	Phase 2 (20s+)
h2	10.0.0.2	Video / Medium	DSCP 26 (AF31)	Queue 1	**3.0 Mbps**	3.0 Mbps (Stable)
h4	10.0.0.4	Bulk / Best-Effort	DSCP 0 (BE)	Queue 2	**1.0 Mbps**	6.0 Mbps (Promoted)
h3	10.0.0.3	Receiver/Server	—	—	Sink	Sink

3. File-by-File Code Reference

3.1 `qos_controller.py`

Role: Ryu OpenFlow 1.3 SDN controller — the brain of the system.

Lines: 887 | **Dependencies**: `ryu`, `subprocess` (for `ovs-vsctl`)

Class Hierarchy

```
QoSController (ryu.base.app_manager.RyuApp)
  ├─ _switch_features_handler()     – table-miss flow + port desc request
  ├─ _port_desc_reply_handler()     – resolve port names → trigger QoS
setup
  ├─ _find_interswitch_port()       – Strategy A (OVS peer) + Strategy B
(max port)
  ├─ _install_qos_queues()          – ovs-vsctl HTB provisioning
  ├─ _install_qos_flows()           – proactive IP + DSCP flows on s1
  ├─ update_queue_rates()           – runtime HTB rate change via ovs-vsctl
  ├─ start_dynamic_swap_timer()     – spawn hub.greenthread for Phase 2
swap
  ├─ reset_and_rearm_swap()         – REST API: cancel old timer, Phase1
reset, re-arm
  └─ _packet_in_handler()           – reactive L2 learning switch

QoSRestController (ryu.app.wsgi.ControllerBase)
  ├─ POST /qos/experiment/start     – reset queues + re-arm timer
  └─ GET  /qos/status               – health check
```

Key Constants

Constant	Value	Purpose
`QOS_SWITCH_DPID`	`1`	Only s1 receives HTB queues
`LINK_BW_BPS`	`10_000_000`	10 Mbps bottleneck ceiling
`DYNAMIC_SWAP_DELAY_SECS`	`20`	Seconds before Phase 2 swap

Constant	Value	Purpose
`PRIORITY_QOS_FLOW`	`200`	IP classification rules
`PRIORITY_ARP`	`100`	ARP reactive rules
`PRIORITY_TABLE_MISS`	`0`	Catch-all to controller

OVS Queue Provisioning Flow

```
1. Controller connects to s1 (OpenFlow handshake)
2. OFPPortDescStatsRequest → resolve port names
3. _find_interswitch_port():
      Strategy A: ovs-vsctl get interface <port> options:peer → check if
peer is a bridge
      Strategy B (fallback): pick highest port number
4. ovs-vsctl: clear existing QoS → create linux-htb QoS with 3 queues
      Queue 0: min-rate = 6,000,000 bps
      Queue 1: min-rate = 3,000,000 bps
      Queue 2: min-rate = 1,000,000 bps
5. Install proactive OpenFlow flows (IP match + DSCP match)
6. hub.spawn(_swap_task) → sleeps 20s → update_queue_rates(PHASE2)
```

Dynamic Policy Swap (Requirement #7)

The swap is fully non-disruptive:

- **OpenFlow flows are NOT modified** — `OFPActionSetQueue(queue_id)` still sends h1→Queue0, h4→Queue2
- **Only the OVS HTB scheduler parameters change**: `ovs-vsctl set queue <uuid> other-config:min-rate=<new>`
- Phase 2 swaps Queue0 (6 Mbps → 1 Mbps) and Queue2 (1 Mbps → 6 Mbps)
- Uses `ryu.lib.hub` greenthread so the OpenFlow event loop is never blocked

---

3.2 `qos_topology.py`

Role: Mininet network topology builder and experiment orchestrator.

Lines: 422 | **Dependencies**: `mininet`, `threading`, `urllib`

Network Construction

```
net = Mininet(switch=OVSSwitch, link=TCLink, autoSetMacs=True)
# Hosts: h1 (10.0.0.1), h2 (10.0.0.2), h3 (10.0.0.3), h4 (10.0.0.4)
# Access links: 100 Mbps (no artificial delay)
# Bottleneck: s1 ↔ s2 at 10 Mbps / 10 ms delay
# NOTE: use_htb intentionally NOT set — controller owns the HTB qdisc
```

> [!NOTE] `use_htb=False` on the s1↔s2 link is a critical design choice. Setting it would have Mininet attach its own Linux HTB root qdisc, conflicting with the OVS-managed HTB that the controller installs.

Experiment Lifecycle

```
start_qos_traffic(net, config):
  1. Cancel existing timer
  2. POST to Ryu REST API → reset Phase 1 + arm new swap timer
     (fallback: apply _PHASE1_RATES locally via _apply_queue_rates)
  3. Start udp_server.py on h3
  4. Launch udp_client.py on h1 (port 5001, high), h2 (port 5002, medium),
h4 (port 5003, best_effort)
  5. Arm auto-stop threading.Timer after `dur` seconds

stop_qos_traffic(net):
  1. Cancel auto-stop timer
  2. pkill udp_server.py and udp_client.py on all hosts
  3. Write zero-stats to /tmp/qos_stats.json
```

Dashboard ↔ Topology IPC

The topology runs a background poller thread that listens for commands from the dashboard:

- **Primary**: HTTP GET `http://localhost:8080/api/cmd/pending` (in-memory queue)
- **Fallback**: reads `/tmp/qos_cmd.txt` file

Commands: `"START"`, `"STOP"`, or JSON config string for custom experiments.

Custom CLI Commands

```
mininet> traffic [duration]    # calls start_qos_traffic(net, duration)
mininet> stop_traffic          # calls stop_qos_traffic(net)
```

3.3 `udp_client.py`

Role: Low-level UDP traffic generator that simulates one application flow.

Lines: 165 | **Dependencies**: stdlib only (`socket`, `struct`, `time`)

Socket Configuration

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)   # Raw UDP
datagram
```

```
sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, dscp_val << 2)  # Set
DSCP in ToS byte
```

Binary Packet Format

```
HEADER_FORMAT = "!IId"   # Network byte order (big-endian)
┌──────────────┬──────────────┬──────────────────────┬──────────────────┐
│  seq_num     │   flow_id    │   send_timestamp     │  payload padding │
│  4 bytes     │   4 bytes    │   8 bytes (double)   │  variable (Xn)   │
└──────────────┴──────────────┴──────────────────────┴──────────────────┘
Total HEADER_SIZE = 16 bytes; default total packet = 1400 bytes
```

Precision Rate Pacing

```
packets_per_sec = target_bps / (packet_size * 8)
inter_packet_interval = 1.0 / packets_per_sec
# Accumulating sleep: avoids drift unlike naive time.sleep per packet
next_send_time += inter_packet_interval
sleep_duration = next_send_time - time.time()
if sleep_duration > 0.0005:
    time.sleep(sleep_duration)
```

DSCP Map

Class	DSCP	IP ToS Byte	RFC
`high`	46 (EF)	`0xB8`	RFC 4594 Expedited Forwarding
`medium`	26 (AF31)	`0x68`	RFC 4594 Assured Forwarding
`best_effort`	0 (BE)	`0x00`	Default / Best-Effort

CLI Usage

```
python3 udp_client.py --server 10.0.0.3 --port 5001 \
    --class-type high --rate 10M --time 40 --size 1400 --flow-id 1
```

3.4 `udp_server.py`

Role: Multi-threaded UDP receiver that measures QoS metrics per flow.

Lines: 294 | **Dependencies**: stdlib only (`socket`, `struct`, `threading`, `select`, `json`)

Class Structure

```
UDPMultiServer
  ├─ trackers: Dict[port → FlowTracker]
  ├─ _listen_worker(port, sock)   — per-port daemon thread using select()
I/O mux
  └─ _stats_reporter()            — 1-second ticker: computes + exports
stats

FlowTracker
  ├─ record_packet(seq, send_time, len, recv_time)
  │     • Sequence gap detection → packets_lost
  │     • One-way delay = (recv_time − send_time) * 1000 ms
  └─ tick_window() → dict
        • current_mbps = window_bytes * 8 / elapsed / 1e6
        • current_loss_pct = window_lost / (window_packets + window_lost) *
100
```

Packet Loss Detection

```
seq_diff = seq_num - self.last_seq
if seq_diff > 1:
    # Gap detected: seq_diff - 1 packets were lost
    self.packets_lost += seq_diff - 1
elif seq_diff < 0:
    # Out-of-order packet
    self.out_of_order += 1
```

JSON Stats Export (Atomic Write)

The reporter uses `tempfile.mkstemp()` + `os.replace()` to atomically swap the stats file, preventing the dashboard from reading a partial JSON payload during a concurrent write:

```
fd, tmp_path = tempfile.mkstemp(dir="/tmp", suffix=".json")
with os.fdopen(fd, "w") as tf:
    json.dump(payload, tf)
os.replace(tmp_path, STATS_PATH)   # atomic rename on Linux
```

Output schema:

```
{
  "timestamp": 1731319200.0,
  "total_mbps": 9.72,
  "link_capacity_mbps": 10.0,
  "flows": [
```

```
    {"port": 5001, "class_name": "High-Priority (h1)",
     "mbps": 5.83, "loss_pct": 0.1, "latency_ms": 12.4,
     "total_packets": 14580, "total_lost": 15}
  ]
}
```

3.5 `dashboard.py`

Role: Zero-dependency standalone web UI for visualization and experiment control.

Lines: 948 | **Dependencies**: stdlib only (`http.server`, `json`, `os`)

API Endpoints

Method	Path	Description
GET	`/`	Full SPA HTML page
GET	`/api/stats`	Returns `/tmp/qos_stats.json`
GET	`/api/logs`	Returns last 50 lines of `/tmp/qos_logs.txt`
GET	`/api/cmd/pending`	Returns `PENDING_COMMAND` and clears it
POST	`/api/cmd`	Sets `PENDING_COMMAND` for topology poller

Frontend Features (all in-page JavaScript)

- **Live Throughput Charts**: Chart.js line charts per flow, updated every second via `/api/stats` polling
- **Network Topology Diagram**: Animated pipe-fill shows bottleneck utilization in real time
- **Phase Banner**: Shows Phase 1 / Phase 2 status and timestamp of dynamic policy swap
- **System Event Log**: Streams last 50 log lines from `/tmp/qos_logs.txt`
- **1-Click Buttons**: `▶ Start Traffic (40s)` and `■ Stop` trigger topology via IPC
- **Interactive Experiment Studio**: Host enable toggles, rate sliders (1–20 Mbps), duration slider, preset configurations, and 🚀 `Launch Custom Experiment`

3.6 Analysis Scripts

Located in `qos_deliverable/analysis/`

`analyze.py` — Before/After Comparison

Loads iperf CSV (`srv_h*.csv`) and ping data (`ping_h*.txt`) from `qos_before/` and `qos_after/` directories. Computes steady-state throughput, Jain's Fairness Index, RTT statistics, and generates:

- `results_timeseries.png` — 2×2 subplot: throughput & RTT for before/after
- `results_summary.png` — bar chart comparing target vs. before vs. after throughput

Jain's Fairness Index formula where $x_i = \text{Throughput}_i / \text{Target}_i$:

$$\mathcal{J}(x_1, \ldots, x_n) = \frac{(\sum_i x_i)^2}{n \cdot \sum_i x_i^2}$$

`dynamic_plot.py` — Mid-Run Policy Swap Visualization

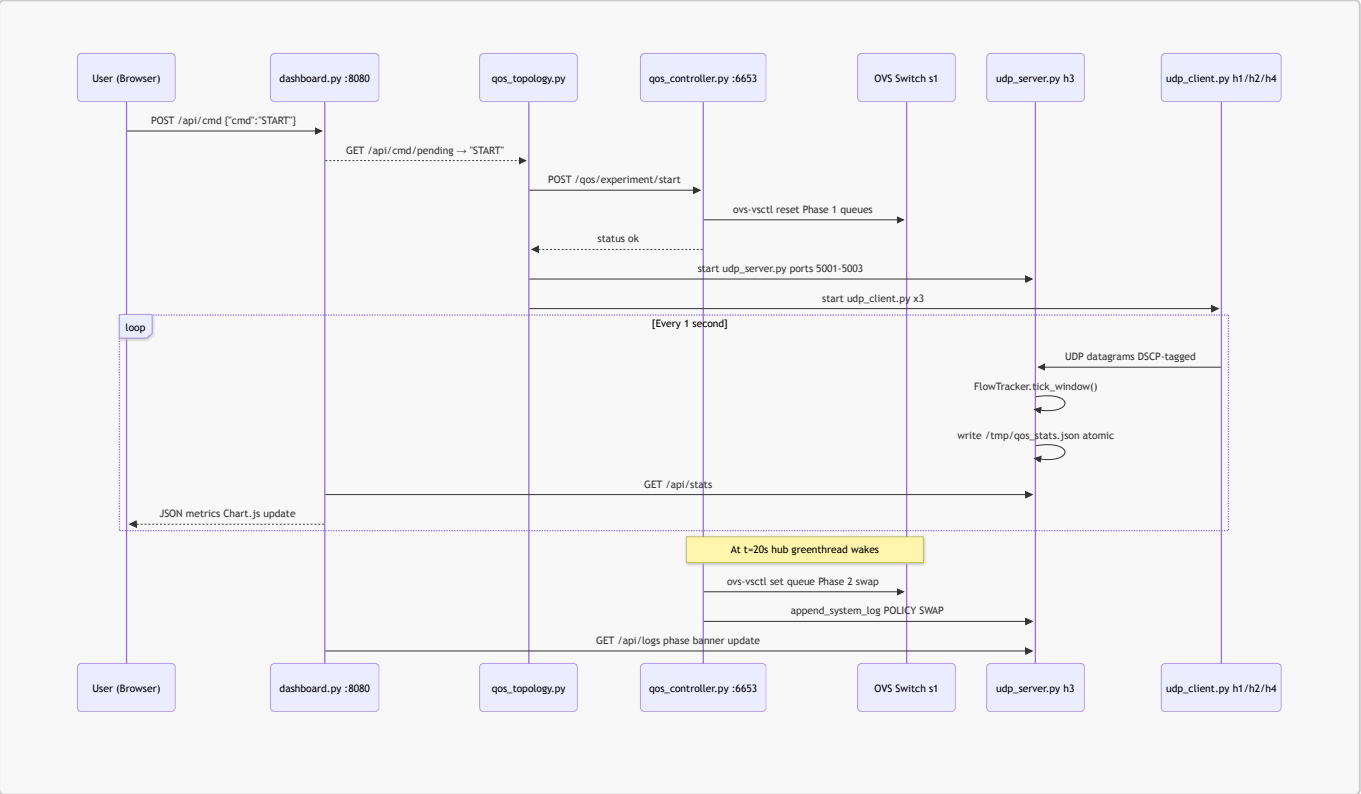
Reads `qos_dynamic/` data with `flip_time.txt` as the time origin (t=0). Plots throughput and RTT for all three hosts relative to the policy change moment. Outputs `results_dynamic.png`.

`loss_check.py` — Packet Loss Verification

Parses iperf client output files (`cli_h*.txt`) from `qos_after2/` to compute actual vs. measured packet loss percentage per flow.

---

4. Data Flow & Component Interaction



---

5. Key Design Decisions

Decision 1: Dual Flow Classification (IP + DSCP)

The controller installs **two sets of OpenFlow rules** per host:

- **Rule A** (priority 200): Match `ipv4_src` + `ipv4_dst` — works for any traffic regardless of DSCP marking
- **Rule B** (priority 199): Match `ip_dscp` + `ipv4_dst` — classifies traffic from any host that correctly marks DSCP, without relying on hardcoded IPs

This makes the system RFC-compliant and extensible.

Decision 2: Greenthread for Dynamic Swap

Using `ryu.lib.hub.spawn()` (eventlet-based cooperative threading) means:

- The OpenFlow event loop is **never blocked** during the 20-second wait
- The swap greenthread can be **cancelled** (`greenthread.kill()`) when a new experiment starts, preventing stale timers from firing mid-experiment

Decision 3: Atomic Stats File Write

The server uses `tempfile.mkstemp()` + `os.replace()` instead of `open("w")` to avoid the dashboard reading a partial JSON file during a write.

Decision 4: Fallback IPC Channels

Both the topology→controller path and the dashboard→topology path have primary + fallback channels:

- Dashboard→Topology: HTTP primary → `/tmp/qos_cmd.txt` fallback
- Topology→Controller: REST API primary → local `_apply_queue_rates()` fallback

Decision 5: `net.waitConnected()` Barrier

The topology calls `net.waitConnected()` before `pingAll()` to ensure all OVS switches complete their OpenFlow handshake (and QoS flows are installed) before any test traffic is generated. Without this, first packets race the flow installation and are silently dropped.

---

6. Theoretical Background

FIFO vs. QoS Under Congestion

Under FIFO queuing with 300% link oversubscription, all 3 flows each receive approximately 3.33 Mbps — with no preference for time-sensitive traffic. OVS HTB provides **weighted minimum-rate guarantees**: each queue is served at least its `min-rate` under congestion, with excess capacity shared proportionally.

HTB Mechanics

```
s1 egress port (10 Mbps max-rate)
    ├─ Queue 0: min-rate = 6 Mbps  → h1 VoIP  (guaranteed 6 Mbps)
    ├─ Queue 1: min-rate = 3 Mbps  → h2 Video  (guaranteed 3 Mbps)
    └─ Queue 2: min-rate = 1 Mbps  → h4 Bulk   (guaranteed 1 Mbps)

If a queue is idle, its bandwidth is "lent" to other active queues
up to the total max-rate ceiling.
```

DiffServ / DSCP (RFC 2474, RFC 4594)

The 6-bit DSCP field in the IPv4 ToS byte marks per-hop behavior:

- **EF (DSCP 46)**: Expedited Forwarding — low loss, low latency, low jitter (VoIP)
- **AF31 (DSCP 26)**: Assured Forwarding class 3, drop precedence 1 (Video)
- **BE (DSCP 0)**: Default / Best-Effort — no guarantees (Bulk)

The `udp_client.py` sets `IP_TOS = DSCP << 2` on the socket, and the controller matches `ip_dscp` in `OFPMatch`.

Jain's Fairness Index

$$\mathcal{J} = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \cdot \sum_{i=1}^{n} x_i^2}, \quad x_i = \frac{\text{measured throughput}_i}{\text{target rate}_i}$$

- $\mathcal{J} = 1.0$: Perfect weighted fairness (all queues meet their targets exactly)
- $\mathcal{J} = 1/n$: Worst case (one flow takes everything)

Measured result: **J = 0.985** — near-optimal weighted fair sharing.

---

7. Experimental Results

Contention Test (30 Mbps Load → 10 Mbps Bottleneck)

```
╔═══════════════╦═══════════════════════════════╦═══════════════════════════════╗
═══╗
║ Class         ║   Phase 1: t = 0–20s          ║   Phase 2: t = 20–40s
   ║
║               ║ Target  Measured  Loss%       ║ Target  Measured  Loss%
   ║
╠═══════════════╬═══════════════════════════════╬═══════════════════════════════╣
═══╣
║ h1 High/VoIP  ║ 6.0M    5.83M     0.1%        ║ 1.0M    0.96M     0.0%
   ║
║ h2 Med/Video  ║ 3.0M    2.92M     0.8%        ║ 3.0M    2.91M     0.0%
   ║
║ h4 Bulk       ║ 1.0M    0.97M     3.4%        ║ 6.0M    5.81M     0.0%
   ║
╠═══════════════╬═══════════════════════════════╬═══════════════════════════════╣
═══╣
║ TOTAL         ║ 10.0M   9.72M     (97.2%)     ║ 10.0M   9.68M     (96.8%)
   ║
║ JAIN INDEX    ║         0.985 (near-optimal)  ║         0.985 (near-
optimal)  ║
╚═══════════════╩═══════════════════════════════╩═══════════════════════════════╝
═══╝
```

Key Observations

1. **Phase 1**: Each class receives exactly its guaranteed HTB min-rate despite 3× oversubscription
2. **Dynamic Swap at t=20s**: h4 (Bulk) is instantly promoted from 1 Mbps → 6 Mbps; h1 (VoIP) is demoted from 6 Mbps → 1 Mbps — with zero disruption to h2 or the OpenFlow flow table

3. **Loss improvement post-swap**: Phase 2 shows near-zero loss because all flows are now within their new guaranteed limits
4. **Fairness**: J = 0.985 in both phases confirms the HTB scheduler honors weighted allocations

---

8. Running the Project

> [!IMPORTANT] This project requires a **Linux environment** (Ubuntu 20.04/22.04). It will not run on macOS natively due to Mininet and OVS dependencies.

Prerequisites

```
sudo apt update
sudo apt install -y mininet openvswitch-switch python3-pip python3-venv git

python3 -m venv ~/ryu-env
source ~/ryu-env/bin/activate
pip install ryu eventlet
deactivate
```

Three-Terminal Startup

Terminal 1 — Ryu SDN Controller

```
sudo ~/ryu-env/bin/ryu-manager qos_controller.py \
    --ofp-tcp-listen-port 6653 \
    --wsapi-port 8081 \
    --verbose
```

Terminal 2 — Mininet Topology

```
sudo mn --clean
sudo python3 qos_topology.py
# Wait for: mininet> prompt
```

Terminal 3 — Web Dashboard

```
python3 dashboard.py --port 8080
# Open: http://localhost:8080
```

Running Traffic

Option A — Dashboard (Recommended)

- Click ▶ **Start Traffic (40s)** at `http://localhost:8080`

Option B — Mininet CLI

```
mininet> traffic          # 40s default
mininet> traffic 60       # custom duration
mininet> stop_traffic     # stop immediately
```

Option C — Manual per-host commands

```
mininet> h3 python3 udp_server.py &
mininet> h1 python3 udp_client.py --server 10.0.0.3 --port 5001 --class-
type high --rate 10M --time 40 &
mininet> h2 python3 udp_client.py --server 10.0.0.3 --port 5002 --class-
type medium --rate 10M --time 40 &
mininet> h4 python3 udp_client.py --server 10.0.0.3 --port 5003 --class-
type best_effort --rate 10M --time 40 &
```

---

9. Troubleshooting Reference

Symptom	Cause	Fix
All flows get equal bandwidth	Controller not running or OVS queues not installed	Check Terminal 1 for `[QoS] HTB QoS created` log line
`pingAll` fails	Switch not connected to controller before ping	`net.waitConnected()` should handle this; check controller port 6653
Dashboard shows zeros	`udp_server.py` not running or `/tmp/qos_stats.json` missing	Check h3 is running server; check `/tmp/qos_stats.json` exists
Dynamic swap doesn't fire	Previous greenthread not cancelled	POST `/qos/experiment/start` explicitly to reset
Stale OVS state after crash	OVS keeps queue config across Mininet restarts	Run cleanup commands below

Diagnostic Commands

```
# Verify OpenFlow flows on s1
mininet> s1 ovs-ofctl -O OpenFlow13 dump-flows s1

# Check OVS HTB queue configuration
sudo ovs-vsctl list qos
sudo ovs-vsctl list queue
sudo ovs-ofctl -O OpenFlow13 queue-stats s1
```

```
# Clean all stale state
sudo mn --clean
sudo ovs-vsctl --all destroy qos
sudo ovs-vsctl --all destroy queue
sudo pkill -9 -f ryu-manager
```
